

CAWSR: Carla-AutoWare Scenario Runner

David Gasinski¹, Olek Osikowicz¹, Gwilym Rutherford¹, and
Donghwan Shin¹

¹ The University of Sheffield

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#))

Summary

CAWSR (CARLA-AutoWare-Scenario Runner) facilitates the simulation-based testing of the open-source autonomous driving system, Autoware, within CARLA, the state-of-the-art open-source driving simulator. Building on existing tools, this project introduces a research-oriented testing framework for the execution of complex driving scenarios, as well as supporting the implementation of a wide range of verification strategies.

Statement of Need

Verifying Autonomous Driving Systems (ADS) is a critical step before they can be deployed. However, relying only on real-world testing is too expensive, inefficient, and potentially dangerous. Consequently, simulation-based testing has become essential, allowing researchers to safely test driving agents against critical situations at scale. Among these tools, CARLA ([Dosovitskiy et al., 2017](#)) has become the de-facto standard in the research community due to its rich ecosystem of open-source tools, benchmarks, and documentation.

Currently, the standard for evaluating ADS in CARLA is the CARLA Leaderboard and its engine, Scenario Runner (SR) ([CARLA, 2025](#)). This framework is typically used to test “black-box” driving agents, such as ML-based systems which expose only sensor-level inputs and driving control outputs. By running a set of predefined, challenging driving scenarios, researchers can systematically assess agent performance using common metrics like driving score, infractions, and route completion. However, applying this testing framework to industry-grade ADS, such as Autoware ([Kato et al., 2018](#)) or Apollo ([Baidu, 2017](#)), remains difficult. Although communication bridges exist between CARLA and these systems ([Guardstrikelab, 2023](#); [Kaljavesi et al., 2024](#)), they lack native support for scenario execution engines, which limits their utility for scenario-based testing.

This gap has created a significant bottleneck for the research community. Previously, researchers developing scenario generation algorithms mainly relied on combining Apollo with the LGSVL simulator ([Rong et al., 2020](#)). However, LGSVL is now outdated, with official support ending in January 2022. This leaves many researchers without a suitable industry-grade “subject” for evaluating their algorithms.

CAWSR aims to bridge this gap by enabling the evaluation of Autoware in complex driving scenarios within CARLA. By building on the established CARLA platform, this work provides a modern replacement for the outdated Apollo/LGSVL workflow. It also allows Autoware to be directly compared with state-of-the-art research agents on the CARLA Leaderboard.

Effective ADS verification requires the ability to systematically explore the operational design domain. To support this, CAWSR provides a flexible interface for algorithmic scenario generation. This facilitates a wide range of verification strategies based on common metrics, such as the CARLA Leaderboard's driving score ([CARLA Team, 2024](#)).

41 Lastly, it is worth noting that simulators can often introduce unintended nondeterminism,
42 which leads to inconsistent test results (Chance et al., 2022; Osikowicz et al., 2025). Therefore,
43 CAWSR is designed to minimise such nondeterminism throughout the evaluation pipeline.

44 State of the Field

45 Simulation-based testing of Autonomous Vehicles spans several tools, but CAWSR occupies a
46 unique niche at the intersection of industry-grade ADS integration, scenario-based testing, and
47 programmatic scenario generation.

48 **Simulators:** CARLA (Dosovitskiy et al., 2017) and SR (CARLA, 2025) play a key role as the
49 de-facto standard for open-source ADS research. However, their driving agent architecture
50 inherently assumes a black-box model (for a good reason to encapsulate the ADS details),
51 lacking the necessary mechanism to manage the complex message passing of a ROS2-based
52 system like Autoware.

53 **Autoware Integrations:** The CARLA-Autoware Bridge (Kaljavesi et al., 2024) provides low-level
54 data translation between CARLA and ROS2, serving as a vital dependency for our work.
55 However, it does not provide any scenario execution capabilities. PCLA (Tehrani et al., 2025)
56 simplifies the deployment of multiple pretrained ML-based driving agents across multiple
57 CARLA versions, but abstracts away the deep simulator-agent integration required for modular
58 ADS like Autoware.

59 Recent concurrent developments, such as the autoware-carla leaderboard (Kaljavesi et al.,
60 2026), have also recognised this gap, introducing support for executing predefined Leaderboard
61 scenarios with Autoware in CARLA. While these efforts confirm the urgent need, they optimise
62 mainly for static benchmark execution. CAWSR diverges in its fundamental design by exposing
63 a programmatic interface tailored for algorithmic, iterative scenario generation, which is a
64 strict prerequisite for systematic ADS testing research.

65 **Build vs. Contribute:** The state of the field presents a clear rationale for CAWSR as a new tool
66 rather than a contribution to an existing project. Extending SR directly to support Autoware
67 would require fundamentally rewriting its core agent model, breaking backwards compatibility
68 for ML-based users. Therefore, building CAWSR as a standalone orchestration layer that
69 leverages SR's underlying behavior trees, while maintaining a specialised ROS2 agent interface,
70 presents the most viable scholarly contribution.

71 Software Design

72 CAWSR is a fully synchronous testing framework that directly integrates the CARLA simulator,
73 Scenario Runner (as the scenario executor), and Autoware (as the System Under Test) to
74 facilitate autonomous driving testing research. The tool is distributed as a containerized
75 deployment using Docker to manage complex dependencies and simplify the setup process.
76 Currently, two modes of operation are supported:

- 77 1. *Scenario Generation Mode:* Enables the dynamic generation and execution of scenarios
78 (e.g. iterative scenario generation) provided by a user-defined algorithm. This is
79 particularly useful for assessing the performance of new simulation-based ADS testing
80 techniques.
- 81 2. *Benchmark Mode:* Allows the execution of a predefined set of scenario definitions provided
82 by the user. This is useful for standardised evaluations and comparisons between different
83 driving agents.

84 The evaluation pipeline is engineered to be fully synchronous, minimising unintentional non-
85 determinism to facilitate reproducible results. However, it is noted that minor variations may

86 still persist due to inherent non-determinism in upstream dependencies, such as the driving
87 simulator or the driving agent itself (Chance et al., 2022; Osikowicz et al., 2025).

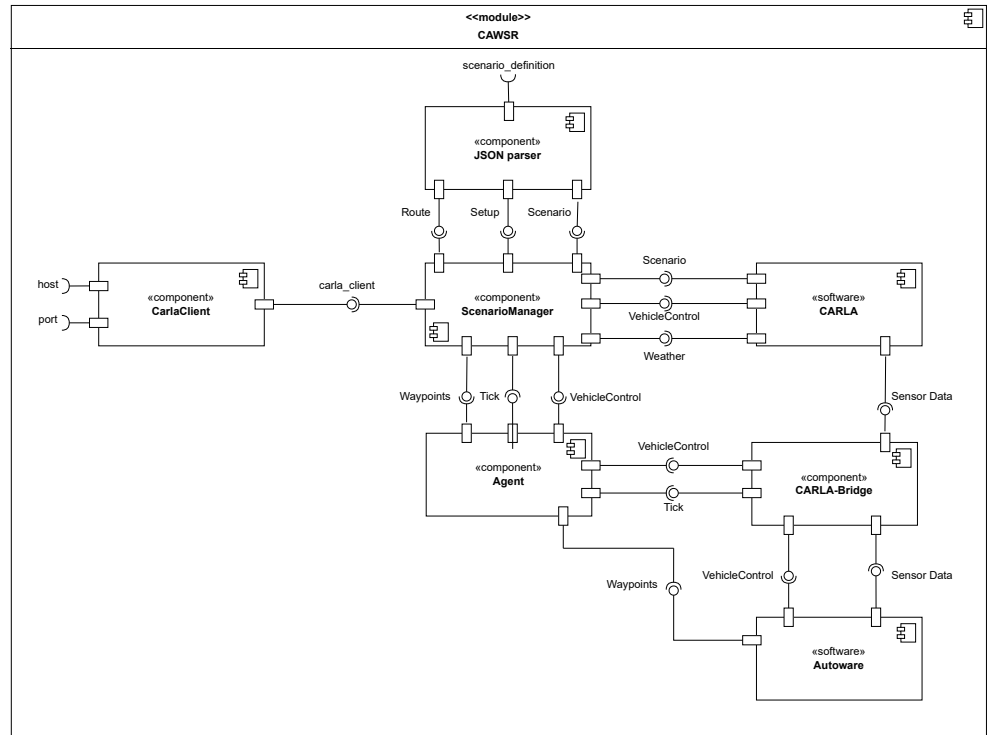


Figure 1: Internal component diagram of CAWSR.

88 Figure 1 illustrates the CAWSR architecture and its fundamental components. The framework
89 operates through four primary modules:

- 90 ■ **CarlaClient:** A native CARLA PythonAPI class that establishes a TCP connection
91 (via host IP and port). It serves as the framework's exclusive interface for extracting
92 simulation data and spawning entities.
- 93 ■ **JSON Parser:** Translates the *scenario_definition* (see Figure 2) into a Behavior Tree
94 (BT). It utilises Scenario Runner's *Atomic Behaviours* and *Atomic Conditions* as modular
95 primitives to define discrete actions (e.g., spawning pedestrians) and logic triggers.
- 96 ■ **ScenarioManager** (CARLA, 2025): Orchestrates the simulation loop by evaluating the
97 BT to update actor states and triggering CARLA simulation ticks. Execution terminates
98 based on CARLA Leaderboard criteria (CARLA Team, 2024), as summarised in Table 1.
99 Post-execution, the module calculates the Driving Score (DS) according to the official
100 leaderboard metrics.
- 101 ■ **Agent and CarlaBridge:** The Agent manages the ROS2 connection to Autaware. At
102 each timestep, the CarlaBridge (Kaljavesi et al., 2024) transforms CARLA snapshots
103 and sensor data into the Autaware coordinate system. Autaware processes these inputs
104 to issue control commands, which the Agent then applies to the ego vehicle.

Table 1: Termination Criteria of each scenario within CAWSR.

Termination Criteria	Description
Route_Completion	Agent reached the end of the route.
Actor_Blocked	Agent is blocked, not moving for 180s.
Simulation_Timeout	No client-server communication established (30s).

To facilitate development, we introduce a new domain model for the definition of route-based scenarios within CARLA, described in Figure 2, alongside a JSON implementation. This model is based on the format introduced by Scenario Runner, facilitating support between both frameworks.

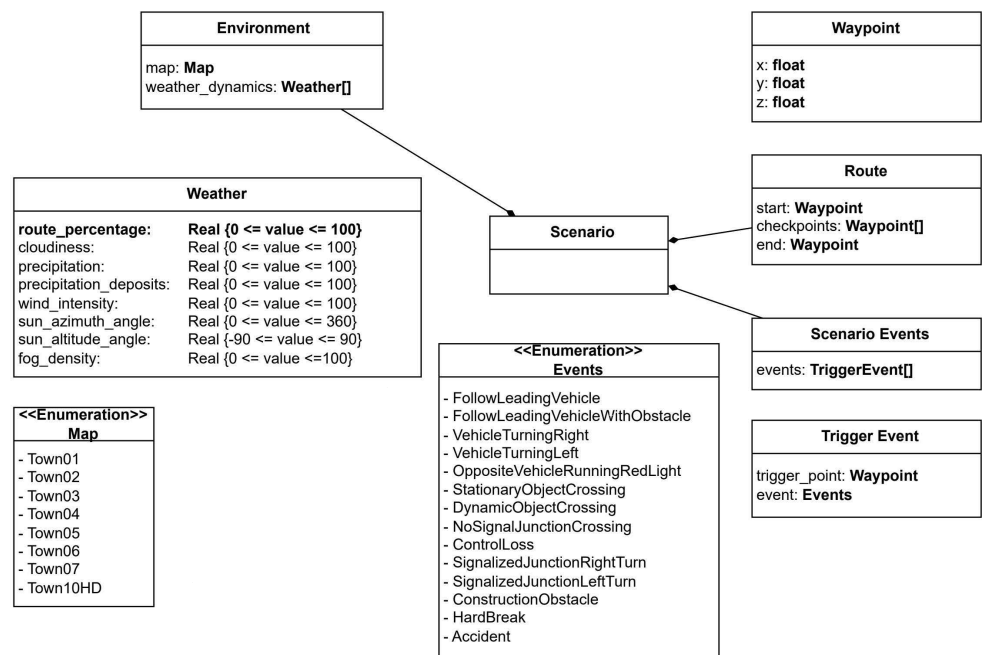


Figure 2: Scenario definition domain model.

Research Impact

CAWSR provides a significant research impact by bridging the gap between the widely-used CARLA simulator and the industry-grade Autoware ADS. It addresses a critical bottleneck in the ADS testing research community by offering a modern replacement for the outdated Apollo/LGSVL workflow, which has lacked official support since 2022.

It enhances research reproducibility by implementing a fully synchronous evaluation pipeline that minimises unintended nondeterminism in simulation-based testing. To ensure community readiness and ease of use, it is distributed as a Docker container.

Furthermore, by adopting CARLA Leaderboard metrics, CAWSR enables researchers to directly compare Autoware with other state-of-the-art driving agents in the CARLA environment. It provides an essential foundation for the systematic evaluation of various testing approaches for ADS.

AI Usage Disclosure

Generative AI tools were used in this work solely to support high-level research concepts and structural ideas. All software implementation, including the source code, architecture, and deployment scripts, was authored entirely by the researchers without AI assistance.

Acknowledgements

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No. RS-2025-02218761, 50%) and by the Engineering and Physical Sciences Research Council (EPSRC) [EP/Y014219/1].

References

- Baidu. (2017). *Apollo: Open Source Autonomous Driving*. <https://github.com/ApolloAuto/apollo>.
- CARLA. (2025). *Scenario Runner: Traffic Scenario Definition and Execution Engine for CARLA*. GitHub; https://github.com/carla-simulator/scenario_runner. https://github.com/carla-simulator/scenario_runner
- CARLA Team. (2024). *Get started with leaderboard 2.0. CARLA autonomous driving leaderboard*. https://leaderboard.carla.org/get_started_v2_0/
- Chance, G., Ghobrial, A., McAreavey, K., Lemaignan, S., Pipe, T., & Eder, K. (2022). On determinism of game engines used for simulation-based autonomous vehicle verification. *IEEE Transactions on Intelligent Transportation Systems*, 23(11), 20538–20552. <https://doi.org/10.1109/TITS.2022.3177887>
- Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., & Koltun, V. (2017). CARLA: An open urban driving simulator. *Proceedings of the 1st Annual Conference on Robot Learning*, 1–16.
- Guardstrikelab. (2023). *carla_apollo_bridge: Data and Control Bridge for Apollo and Carla*. GitHub; https://github.com/guardstrikelab/carla_apollo_bridge.
- Kaljavesi, G., Kerbl, T., Betz, T., Mitkovskii, K., & Diermeyer, F. (2024). *CARLA-autoware-bridge: Facilitating autonomous driving research with a unified framework for simulation and module development*. 224–229. <https://doi.org/10.1109/iv55156.2024.10588623>
- Kaljavesi, G., Majstorovic, D., Clement, M., & Diermeyer, F. (2026). *Empirical mapping of technical bottlenecks in autonomous driving: A cross-platform evaluation of simulated and real-world performance*.
- Kato, S., Tokunaga, S., Maruyama, Y., Maeda, S., Hirabayashi, M., Kitsukawa, Y., Monroy, A., Ando, T., Fujii, Y., & Azumi, T. (2018). Autoware on board: Enabling autonomous vehicles with embedded systems. *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPs)*, 287–296. <https://doi.org/10.1109/iccps.2018.00035>
- Osikowicz, O., McMinn, P., & Shin, D. (2025). Empirically evaluating flaky tests for autonomous driving systems in simulated environments. *2025 IEEE/ACM International Flaky Tests Workshop (FTW)*, 13–20. <https://doi.org/10.1109/FTW66604.2025.00009>
- Rong, G., Shin, B. H., Tabatabaee, H., Lu, Q., Lemke, S., Možeiko, M., Boise, E., Uhm, G., Gerow, M., Mehta, S., Agafonov, E., Kim, T. H., Sterner, E., Ushiroda, K., Reyes, M., Zelenkovsky, D., & Kim, S. (2020). LGSVL simulator: A high fidelity simulator for autonomous driving. *2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC)*, 1–6. <https://doi.org/10.1109/ITSC45102.2020.9294422>

- 164 Tehrani, M. J., Kim, J., & Tonella, P. (2025). PCLA: A framework for testing autonomous
165 agents in the CARLA simulator. *Proceedings of the 33rd ACM International Conference on*
166 *the Foundations of Software Engineering*, 1040–1044. [https://doi.org/10.1145/3696630.](https://doi.org/10.1145/3696630.3728577)
167 [3728577](https://doi.org/10.1145/3696630.3728577)

DRAFT